Freestanding Library: algorithm, numeric, and random

Document number: P2976R1
Date: 2024-05-05
Reply-to: Ben Craig <ben dot craig at gmail dot com>
Audience: Library Working Group

Changes from previous revisions

R1

- Rebased to N4981
- Update `__cpp_lib_freestanding_numeric` instead of adding it.
- Tweaked [freestanding.item] wording.
- Added `is_execution_policy` in `<execution>` to freestanding.
- Marking parallel algorithms in `<memory>`, `<algorithm>`, and `<numeric>` as freestanding-deleted.
- Drive by improvement of the `<random>` streaming operator specification to make "Returns" explicit.

R0

First revision!

Introduction

This proposal is part of a group of papers aimed at improving the state of freestanding.

At a high level, this proposal adds most of the facilities from `<algorithm>`, `<numeric>`, and `<random>` that don't require floating point or `ExecutionPolicy`.

Implementations of these often rely on facilities like `memcpy`, which is made freestanding by P2338 "Freestanding Library: Character primitives and the C library".

Motivation and Scope

On hosted implementations, the included facilities don't directly require heap allocations, don't use system calls, and don't require exceptions. The facilities are generally useful. This combination keeps the burden low for standard library implementations to single source these facilities as both hosted and freestanding.

Design

Parallel Algorithms

The execution policy overloads in `<memory>`, `<algorithm>`, and `<numeric>` are marked as freestanding-deleted. That means that either the functions are `=deleted`, or they have the same semantics as they would have on a

hosted implementation. An argument could be made to support sequential execution, but that provides very little value.

Allocating algorithms

Some algorithms (`stable_sort`, `stable_partition`, and `inplace_merge`) attempt to allocate temporary buffers as a way to reduce the time complexity of the algorithm. Freestanding implementations are not required to have an operational `::operator new`. As a result, these algorithms are not required to be present in freestanding implementations.

In theory, we could include these algorithms with the expectation that the fall-back, worse time complexity is used instead of the complexity that we get from the temporary buffer version of the algorithm. I did not take that approach, as I see it being a subtle performance break. I would much rather have a loud compiler error than a subtle performance break.

Random number iostreams

This paper makes the iostreams streaming requirements optional for random number engines [rand.req.eng] and random number distributions [rand.req.dist] on freestanding implementations. Freestanding doesn't require iostreams to be present, and won't require the random number iostreams operations to be present either. Streaming these objects allows users to save and restore the state of the object in a round-trippable manner. Common usages of the random number facilities that don't involve serialization still work without iostreams support.

Floating point randomness

In many kernel environments, loading, storing, or manipulating floating point values is not allowed in the general case. Using floating point in these areas ends up corrupting user mode floating point state. As a result, freestanding avoids requiring facilities that use floating point types.

The random number facilities in `<random>` are a mix of integer and floating point facilities. All the distributions other than `uniform_int_distribution` rely on floating point math. In addition, the `shuffle_order_engine` and its associated `knuth_b` typedef are specified and implemented in terms of floating point math. `generate_canonical` also deals directly with floating point numbers.

OS dependent random number facilities

Implementing `random_device` in a useful way requires knowledge of the operating environment. While a conforming implementation of `random_device` is allowed to return pseudo-random numbers, actually doing so is user hostile. If users want `random_device` non-deterministic random numbers, and the system cannot supply them, then users would typically prefer a compiler error rather than getting incorrect, deterministic random results.

Due to this dependency on the operating environment, `random_device` is not required to be present in freestanding implementations.

`seed_seq` requires dynamic memory allocation. Dynamic memory allocation is typically considered a facility provided by the operating environment. As a result, `seed_seq` is not required to be present in freestanding implementations.

All of the standard library random engines have constructors taking a seed sequence. Those constructors are templated on the seed sequence, and do not specifically require `seed_seq`.

Experience

These changes have been tested with the libc++ test suite in the Microsoft Windows 11 kernel with the MSVC STL. The binaries produced were scanned for unexpected floating point usage. Usage of exceptions, iostreams, and global `::operator new` and `::operator delete` cause linker errors on that platform.

The C++03 portions of the library have seen extensive field use in kernel drivers produced by NI (formerly National Instruments). Those drivers use a modified version of STLport in kernel drivers built for Microsoft Windows, Linux, and Apple MacOS, as well as for various real time operating systems.

Paul Bendixen has implemented P0829 in a libstdc++ fork. P0829 is (almost) a superset of the facilities in this paper.

Wording

This paper's wording is based on the current working draft N4981.

Changes in [freestanding.item]

5 An entity, deduction guide, or *typedef-name* is a freestanding item if ~~it is~~**its introducing declaration is not followed by a comment that includes *hosted*, and is**:

(5.1) - introduced by a declaration that is a freestanding item,
(5.2) - a member of a freestanding item other than a namespace,
(5.3) - an enumerator of a freestanding item,
(5.4) - a deduction guide of a freestanding item,
(5.5) - an enclosing namespace of a freestanding item,
(5.6) - a friend of a freestanding item,
(5.7) - denoted by a *typedef-name* that is a freestanding item, or
(5.8) - denoted by an alias template that is a freestanding item.

Changes in [compliance]

Drafting note: `<algorithm>`, `<memory>`, and `<numeric>` are already freestanding headers.
Add new rows to the "C++ headers for freestanding implementations" table:

Subclause		Header(s)
[…]	[…]	[…]
?.? [execpol]	**Execution policies**	**`<execution>`**
[…]	[…]	[…]
?.? [rand]	**Random number generation**	**`<random>`**
[…]	[…]	[…]

Changes in [version.syn]

Instructions to the editor:
Add the following macros to [version.syn]:

```
#define __cpp_lib_freestanding_execution 20XXXXL // freestanding, also in <execution>
#define __cpp_lib_freestanding_random 20XXXXL // freestanding, also in <random>
```

Update the integer literal for the following macros:

- `__cpp_lib_freestanding_algorithm`
- `__cpp_lib_freestanding_memory`
- `__cpp_lib_freestanding_numeric`

Changes in [memory.syn]

Instructions to the editor:
Please make the following edit to the comments after each of the following parallel algorithms (i.e. function templates with `ExecutionPolicy` overloads):

```
// freestanding-deleted, see [algorithms.parallel.overloads]
```

- `uninitialized_default_construct`
- `uninitialized_default_construct_n`
- `uninitialized_value_construct`
- `uninitialized_value_construct_n`
- `uninitialized_copy`
- `uninitialized_copy_n`
- `uninitialized_move`
- `uninitialized_move_n`
- `uninitialized_fill`
- `uninitialized_fill_n`
- `destroy`
- `destroy_n`

Changes in [execution.syn]

Instructions to the editor:
Please append a *`// freestanding`* comment to `is_execution_policy` and `is_execution_policy_v`.

Changes in [algorithm.syn]

Instructions to the editor:

Please insert a *`// mostly freestanding`* comment at the beginning of the [algorithm.syn] synopsis.

Please make the following edit to the comments after all of the parallel algorithms (i.e. function templates with `ExecutionPolicy` overloads), other than the three allocating algorithms:

```
// freestanding-deleted, see [algorithms.parallel.overloads]
```

- `stable_sort`
- `stable_partition`
- `inplace_merge`

Please append a `// hosted` comment to all overloads of the following function templates:

- `stable_sort`
- `stable_partition`
- `inplace_merge`

Please **remove** the `// freestanding` comment from the following functions. The markings are now redundant given the `// mostly freestanding` comment at the beginning of the synopsis.

- `fill_n(OutputIterator first, Size n, const T& value)`
- `swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1, ForwardIterator2 first2)`

Changes in [numeric.ops.overview]

Instructions to the editor:

Please insert a `// mostly freestanding` comment at the beginning of the [numeric.ops.overview] synopsis.

Please make the following edit to the comment after each of the parallel algorithms (i.e. function templates with `ExecutionPolicy` overloads):

```
// freestanding-deleted, see [algorithms.parallel.overloads]
```

Please **remove** the `// freestanding` comment from the following functions. The markings are now redundant given the `// mostly freestanding` comment at the beginning of the synopsis.

- `add_sat`
- `sub_sat`
- `mul_sat`
- `div_sat`
- `saturate_cast`

Changes in [rand.synopsis]

Instructions to the editor:
Please append a `// freestanding` comment to the following declarations:

- `uniform_random_bit_generator`
- `minstd_rand0`
- `minstd_rand`
- `mt19937`
- `mt19937_64`
- `ranlux24_base`
- `ranlux48_base`
- `ranlux24`
- `ranlux48`

Drafting note: The omission of `knuth_b` is intentional.
Please append a `// partially freestanding` comment to the following declarations:

- `linear_congruential_engine`
- `mersenne_twister_engine`
- `subtract_with_carry_engine`
- `discard_block_engine`
- `independent_bits_engine`
- `uniform_int_distribution`

Drafting note: The omission of `shuffle_order_engine` is intentional.

Changes in [rand.req.eng]

Table 96: Random number engine requirements [tab:rand.req.eng]

Expression	Return type	Pre/post-condition	Complexity
...	...	...	
~~`os << x`~~	~~reference to the type of `os`~~	~~With `os.`*fmtflags* set to `ios_base::dec|ios_base::left` and the fill character set to the space character, writes to `os` the textual representation of `x`'s current state. In the output, adjacent numbers are separated by one or more space characters. *Postconditions*: The `os.`*fmtflags* and fill character are unchanged.~~	~~O(size of state)~~
~~`is >> v`~~	~~reference to the type of `is`~~	~~With `is.fmtflags` set to `ios_base::dec`, sets `v`'s state as determined by reading its textual representation from `is`. If bad input is encountered, ensures that `v`'s state is unchanged by the operation and calls `is.setstate(ios_base::failbit)` (which may throw `ios_base::failure` ([iostate.flags])). If a textual representation written via `os << x` was subsequently read via `is >> v`, then `x == v` provided that there have been no intervening invocations of `x` or of `v`. *Preconditions*: `is` provides a textual representation that was previously written using an output stream whose imbued locale was the same as that of `is`, and whose type's template specialization arguments~~	~~O(size of state)~~

	~~`charT` and `traits` were respectively the same as those of `is`.~~ ~~*Postconditions*: The `is`.*fmtflags* are unchanged.~~

5 ...

6 **On hosted implementations, the following expressions are well-formed and have the specified semantics.**

`os << x`

7 **_Effects_: With `os`._fmtflags_ set to `ios_base::dec|ios_base::left` and the fill character set to the space character, writes to `os` the textual representation of `x`'s current state. In the output, adjacent numbers are separated by one or more space characters.**

8 **_Postconditions_: The `os`._fmtflags_ and fill character are unchanged.**

9 **_Result_: reference to the type of `os`.**

10 **_Returns_: `os`.**

11 **_Complexity_: O(size of state)**

`is >> v`

12 **_Preconditions_: `is` provides a textual representation that was previously written using an output stream whose imbued locale was the same as that of `is`, and whose type's template specialization arguments `charT` and `traits` were respectively the same as those of `is`.**

13 **_Effects_: With `is`._fmtflags_ set to `ios_base::dec`, sets `v`'s state as determined by reading its textual representation from `is`. If bad input is encountered, ensures that `v`'s state is unchanged by the operation and calls `is.setstate(ios_base::failbit)` (which may throw `ios_base::failure` ([iostate.flags])). If a textual representation written via `os << x` was subsequently read via `is >> v`, then `x == v` provided that there have been no intervening invocations of `x` or of `v`.**

14 **_Postconditions_: The `is`._fmtflags_ are unchanged.**

15 **_Result_: reference to the type of `is`.**

16 **_Returns_: `is`.**

17 **_Complexity_: O(size of state)**

Changes in [rand.req.dist]

Table 97: Random number distribution requirements [tab:rand.req.dist]

Expression	Return type	Pre/post-condition	Complexity
...	...	...	...
~~`os << x`~~	~~reference to the~~	~~Writes to `os` a textual representation for the parameters and the additional internal data of `x`.~~	

	~~type of `os`~~	~~*Postconditions*: The `os.`*fmtflags* and fill character are unchanged.~~
~~`is >> d`~~	~~reference to the type of `is`~~	~~Restores from `is` the parameters and additional internal data of the lvalue `d`. If bad input is encountered, ensures that `d` is unchanged by the operation and calls `is.setstate(ios_base::failbit)` (which may throw `ios_base::failure` ([iostate.flags])).~~ ~~*Preconditions*: `is` provides a textual representation that was previously written using an `os` whose imbued locale and whose type's template specialization arguments `charT` and `traits` were the same as those of `is`.~~ ~~*Postconditions*: The `is.`*fmtflags* are unchanged.~~

...

11 **On hosted implementations, the following expressions are well-formed and have the specified semantics.**

`os << x`

12 ***Effects*: Writes to `os` a textual representation for the parameters and the additional internal data of `x`.**

13 ***Postconditions*: The `os.`*fmtflags* and fill character are unchanged.**

14 ***Result*: reference to the type of `os`.**

15 ***Returns*: `os`.**

`is >> d`

16 ***Preconditions*: `is` provides a textual representation that was previously written using an `os` whose imbued locale and whose type's template specialization arguments `charT` and `traits` were the same as those of `is`.**

17 ***Effects*: Restores from `is` the parameters and additional internal data of the lvalue `d`. If bad input is encountered, ensures that `d` is unchanged by the operation and calls `is.setstate(ios_base::failbit)` (which may throw `ios_base::failure` ([iostate.flags])).**

18 ***Postconditions*: The `is.`*`fmtflags`* are unchanged.**

19 ***Result*: reference to the type of `is`.**

20 ***Returns*: `is`.**

Changes in [rand.eng.lcong]

Instructions to the editor:
Please append a *// hosted* comment to each `operator<<` and `operator>>` overload in the `linear_congruential_engine` synopsis.

Changes in [rand.eng.mers]

Instructions to the editor:
Please append a *// hosted* comment to each `operator<<` and `operator>>` overload in the `mersenne_twister_engine` synopsis.

Changes in [rand.eng.sub]

Instructions to the editor:
Please append a *// hosted* comment to each `operator<<` and `operator>>` overload in the `subtract_with_carry_engine` synopsis.

Changes in [rand.adapt.disc]

Instructions to the editor:
Please append a *// hosted* comment to each `operator<<` and `operator>>` overload in the `discard_block_engine` synopsis.

Changes in [rand.adapt.ibits]

Instructions to the editor:
Please append a *// hosted* comment to each `operator<<` and `operator>>` overload in the `independent_bits_engine` synopsis.

Changes in [rand.dist.uni.int]

Instructions to the editor:
Please append a *// hosted* comment to each `operator<<` and `operator>>` overload in the `uniform_int_distribution` synopsis.